



ADS08：动态规划（Dynamic Programming）

来源：/home/vincent/Code/ads/Materials/Student Slides/ADS08DynamicProgramming_Stu.ppt

课件范围：S1-S3、S4-S6、S7-S9、S10-S11、S12、S13-S16、S17-S19；共 19 张幻灯片。

正文按原课件顺序整理；S 编号指幻灯片顺序。公式补全、边界修正和原文疑点均明确标注。

本讲路线图（Lecture Roadmap）

消除重复子问题 → Fibonacci 滚动计算 → 矩阵链乘法 → 最优二叉搜索树 → 全源最短路径 → 动态规划设计步骤 → 双装配线 → 适用条件与项目。

1. 从重复递归到动态规划（From Repeated Recursion to Dynamic Programming）[S1-S3]

动态规划（Dynamic Programming, DP）只求解每个子问题一次，并保存结果供后续状态使用。表格替代重复递归，按依赖关系逐步得到最终答案。[S1-S2]

1.1 Fibonacci 的重复子问题（Overlapping Fibonacci Subproblems）[S2-S3]

重要性：A · 必须掌握

本讲的 Fibonacci 数列边界是

$$F(0) = F(1) = 1, \quad F(N) = F(N-1) + F(N-2) \quad (N \geq 2).$$

它从 1, 1, 2, 3, 5, 8, 13, ... 开始，注意与常见的 $F(0) = 0$ 版本不同。[S2-S3]

直接递归在 $N \leq 1$ 时返回 1，否则分别递归求 $F(N-1)$ 与 $F(N-2)$ 。计算 $F(6)$ 时， $F(4)$ 、 $F(3)$ 、 $F(2)$ 等会在不同分支反复出现。[S2-S3]

核心原理（Core Principle）

相同下标的子问题答案相同，但朴素递归每次都重新展开；保存已经计算的状态即可消除重复工作。

1. 朴素递归调用时间满足 $T(N) = T(N-1) + T(N-2) + \Theta(1)$ 。
2. 调用树规模按 Fibonacci 量级增长， $T(N) \geq F(N)$ 说明增长至少是指数性的。[S2]
3. DP 按 $0, 1, 2, \dots, N$ 的顺序处理，每次只用前两个已经得到的答案。[S3]

补充推导：采用固定成本算术模型， $F(N) = \Theta(\varphi^N)$ ，其中 $\varphi = (1 + \sqrt{5})/2$ ，故朴素递归时间为 $\Theta(\varphi^N)$ 。若计算任意大的精确整数，加法的位成本也会增长；本讲的 $O(N)$ 指固定成本算术操作数。

1.2 滚动保存与循环不变量 (Rolling Values and Loop Invariant) [S3]

重要性: A · 必须掌握

```
int Fibonacci(int N) {
    if (N <= 1) return 1;
    int Last = 1, NextToLast = 1, Answer = 1;
    for (int i = 2; i <= N; ++i) {
        Answer = Last + NextToLast;
        NextToLast = Last;
        Last = Answer;
    }
    return Answer;
}
```

循环不变量 (Loop Invariant) 是: 进入下标 i 的迭代前, $\text{Last} = F(i-1)$ 、 $\text{NextToLast} = F(i-2)$ 。

1. **初始化:** $i = 2$ 前分别保存 $F(1), F(0)$ 。
2. **计算:** 相加得到 $F(i)$ 。
3. **更新:** 先把旧 Last 存入 NextToLast, 再将答案写入 Last, 为下一轮保留前两项。
4. **结束:** 最后一轮得到 $F(N)$ 。[S3]

时间为 $O(N)$, 只保留常数个数值, 辅助空间为 $O(1)$ 。[S3; 空间为解释补充]

解释补充: 更新顺序不能颠倒, 否则会丢掉旧的 $F(i-1)$ 。若需要完整数列表, 就不能只留两项。C 的 `int` 会溢出, 应根据输入范围选数值类型。状态只求一次也可以通过记忆化递归 (Memoization) 实现, 不一定必须写成无递归循环。

2. 矩阵链乘法 (Matrix-Chain Multiplication) [S4–S6]

矩阵乘法满足结合律, 改变括号不改变乘积, 却会改变标量乘法次数。目标是选择总代价最小的括号化 (Parenthesization)。[S4]

2.1 括号顺序改变代价 (Cost Depends on Parenthesization) [S4]

重要性: B · 重点理解

给定

$$M_1[10 \times 20], \quad M_2[20 \times 50], \quad M_3[50 \times 1], \quad M_4[1 \times 100].$$

将 $a \times b$ 与 $b \times c$ 矩阵相乘, 需要 abc 次标量乘法, 结果为 $a \times c$ 矩阵。[S4]

核心原理 (Core Principle)

先乘哪些矩阵决定中间矩阵的尺寸; 很大的中间尺寸会使后续乘法昂贵, 因此应比较完整方案的总成本。

1. 顺序 $M_1(M_2(M_3M_4))$:

- M_3M_4 产生 50×100 , 代价 $50 \cdot 1 \cdot 100 = 5000$ 。
- 再左乘 M_2 , 代价 $20 \cdot 50 \cdot 100 = 100000$ 。
- 最后左乘 M_1 , 代价 $10 \cdot 20 \cdot 100 = 20000$ 。
- 总计 125000 次。

2. 顺序 $(M_1(M_2M_3))M_4$:

- M_2M_3 产生 20×1 , 代价 $20 \cdot 50 \cdot 1 = 1000$ 。
- 再左乘 M_1 , 代价 $10 \cdot 20 \cdot 1 = 200$ 。
- 最后右乘 M_4 , 代价 $10 \cdot 1 \cdot 100 = 1000$ 。
- 总计 2200 次。[S4]

解释补充: 括号化允许改变结合顺序, 不允许交换矩阵先后顺序。

2.2 状态与最后一次乘法 (State and Last Multiplication) [S5]

重要性: A · 必须掌握

设 M_i 的尺寸为 $r_{i-1} \times r_i$, m_{ij} 是计算连续链 $M_i \cdots M_j$ 的最小标量乘法次数。单个矩阵不需要乘法, 故 $m_{ii} = 0$ 。[S5–S6]

补充推导: S5 的递推公式未能恢复, 以下依照其状态定义和 S6 的代码补全。

按区间大小分类:

1. **单矩阵:** $m_{ii} = 0$ 。
2. **多个矩阵:** 最后一次乘法必有分割点 L , $i \leq L < j$, 分别计算 $M_i \cdots M_L$ 与 $M_{L+1} \cdots M_j$ 。
3. **合并:** 两个结果尺寸为 $r_{i-1} \times r_L$ 与 $r_L \times r_j$, 代价为 $r_{i-1}r_Lr_j$ 。

因此

$$m_{ij} = \min_{i \leq L < j} (m_{iL} + m_{L+1,j} + r_{i-1}r_Lr_j).$$

核心原理 (Core Principle)

枚举最后一次乘法的位置; 若左右区间不是各自最优, 替换为更便宜方案后结果尺寸不变, 却能降低总成本。

1. 最后一次乘法把有序链切成两个连续区间, 不能出现交叉或乱序子问题。
2. 固定分割点后, 左右区间成本独立相加。
3. 中间结果尺寸只由区间端点决定, 因此替换子方案不会改变最终合并成本。

4. 最优方案必须包含两个最优子方案；枚举所有 L 取最小不会漏掉最优根分割。[S5–S6；证明展开为补充]

不同括号方案数满足 $b_2 = 1, b_3 = 2, b_4 = 5$ ，但区间状态只有 $O(N^2)$ 个。[S5]

补充推导： 以 $b_1 = 1$ 为边界， $b_n = \sum_{L=1}^{n-1} b_L b_{n-L} = C_{n-1}$ ，其中 C_k 为 Catalan 数（Catalan Number）。故 $b_n = \Theta(4^n/n^{3/2})$ ；定点渲染中可读的 $O(4^n/(n\sqrt{n}))$ 与此一致。DP 复用区间最优值，不枚举每个完整括号方案。

2.3 填表、示例与方案恢复（Table Order and Reconstruction）[S6]

重要性：A · 必须掌握

按区间差 $k = j - i$ 从小到大填表：

1. 初始化所有 $m_{ii} = 0$ 。
2. 对 $k = 1, \dots, N - 1$ ，枚举左端点 i ，令 $j = i + k$ 。
3. 将 m_{ij} 初始化为无穷大，枚举 $L = i, \dots, j - 1$ 更新最小值。
4. 最终答案为 m_{1N} 。左右区间都比当前区间短，依赖项已经完成。[S6]

核心原理（Core Principle）

先解短区间再解长区间，使所有候选分割都能读取已完成的子问题；同时记录最优分割点才能恢复括号。

对 S4 的 $r = (10, 20, 50, 1, 100)$ ，填表结果为：

区间	各分割的候选成本	最优成本与分割
[1, 2]	10000	10000, $L = 1$
[2, 3]	1000	1000, $L = 2$
[3, 4]	5000	5000, $L = 3$
[1, 3]	1200, 10500	1200, $L = 1$
[2, 4]	105000, 3000	3000, $L = 3$
[1, 4]	23000, 65000, 2200	2200, $L = 3$

补充推导： 表格按递推展开原尺寸数据。完整区间选 $L = 3$ ，左区间 [1, 3] 选 $L = 1$ ，其右区间是 [2, 3]，恢复为 $(M_1(M_2M_3))M_4$ 。

记 s_{ij} 为最优分割点，恢复按区间分类：

1. $i = j$ ：输出 M_i 。
2. $i < j$ ：输出左括号，递归输出 $[i, s_{ij}]$ 与 $[s_{ij} + 1, j]$ ，再输出右括号。

有 $O(N^2)$ 个状态，每状态枚举 $O(N)$ 个分割，时间为 $O(N^3)$ ；成本表和分割表占 $O(N^2)$ 空间。恢复一个方案访问 $O(N)$ 个子区间。[S6；记录与复杂度展开为补充]

3. 最优二叉搜索树（Optimal Binary Search Tree）[S7–S9]

最优二叉搜索树（Optimal Binary Search Tree, OBST）在键与查询概率固定时，最小化期望查询成本，适用于不发生插入和删除的静态查找（Static Searching）。[S7]

3.1 优化目标与概率（Objective and Probabilities）[S7]

重要性：A · 必须掌握

给定有序键 $w_1 < \dots < w_N$ ，查找 w_i 的概率为 p_i 。本例只给成功查询概率，概率之和为 1。[S7]

键	break	case	char	do	return	switch	void
概率	0.22	0.18	0.20	0.05	0.25	0.02	0.08

根深度为 0、根比较一次时，目标是

$$C(T) = \sum_i p_i (\text{depth}_T(w_i) + 1).$$

解释补充： 该定义与 S8 根贡献 p_k 、S9 单节点表值一致。AVL 树约束高度，并不针对概率加权成本优化；“概率最大者当根”的局部贪心也不保证全局最优。若计入失败查询，需额外给出键间空隙概率，不能直接套用这里的仅成功查询模型。

3.2 区间根枚举与边界（Enumerating Roots and Boundary Cases）[S8]

重要性：A · 必须掌握

为区别键 w_i 与原区间权重 w_{ij} ，这里用 W_{ij} 表示权重：

$$W_{ij} = \sum_{t=i}^j p_t.$$

c_{ij} 为区间 $w_i, \dots, w_j$ 的最优期望成本， r_{ij} 为最优根下标。[S8]

边界分类：

1. 空区间 $i > j$ ： $c_{ij} = 0, W_{ij} = 0$ 。
2. 单节点 $i = j$ ： $c_{ii} = p_i, W_{ii} = p_i, r_{ii} = i$ 。
3. 多节点 $i < j$ ： 枚举根 $k \in [i, j]$ ，左右子树为 $[i, k-1]$ 、 $[k+1, j]$ 。

解释补充： S8 可读文字写 $c_{ii} = 0$ ，但同页公式包含根的 p_k ，S9 单节点成本也逐项为 p_i ，前后冲突。笔记统一采用比较次数模型：单节点 p_i 、空树 0，并保留原文疑点。

核心原理 (Core Principle)

左右子树挂到新根下面后，每个键都多一次比较；新增的概率加权成本恰好是整个区间权重。

固定根 k ，有

$$\begin{aligned} C_k &= p_k + c_{i,k-1} + c_{k+1,j} + W_{i,k-1} + W_{k+1,j} \\ &= W_{ij} + c_{i,k-1} + c_{k+1,j}. \end{aligned}$$

所以

$$c_{ij} = W_{ij} + \min_{i \leq k \leq j} (c_{i,k-1} + c_{k+1,j}),$$

并记录最小值对应的根。[S8；求和、最小化与边界为补充]

1. 根比较一次，贡献 p_k 。
2. 左右子树原成本分别为 $c_{i,k-1}$ 、 $c_{k+1,j}$ 。
3. 子树中每个键的深度加一，贡献它们的概率和。
4. 固定根后，若子树不是最优，则可替换为更优子树并保持键序，降低总成本，故最优树必须使用最优子树。[S8；证明展开为补充]

3.3 七键实例与成本验证 (Seven-Key Example and Cost Check) [S9]

重要性：B · 重点理解

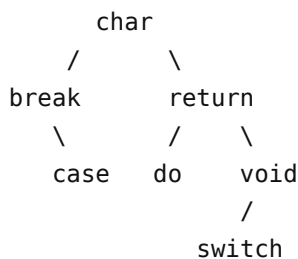
按区间长度递增填表，每区间枚举根。用前缀和 $P_j = \sum_{t=1}^j p_t$ 可在 $O(1)$ 时间得到 $W_{ij} = P_j - P_{i-1}$ 。[S9；实现为补充]

核心原理 (Core Principle)

每个区间比较所有根，而非只比较当前概率；当前根与左右区间总代价一起决定最优结果。

1. **break 到 case**：权重 0.40。break 为根时成本 $0.40 + 0.18 = 0.58$ ；case 为根时为 $0.40 + 0.22 = 0.62$ ，故选 break。
2. **break 到 char**：权重 0.60。三个根的成本为 $0.60 + 0.56 = 1.16$ 、 $0.60 + 0.22 + 0.20 = 1.02$ 、 $0.60 + 0.58 = 1.18$ ，故选 case。
3. **完整七键区间**：最优根为 char，最终 $c_{1,7} = 2.15$ 。[S9；计算展开为补充]

依据根表恢复一棵最优树：



其期望比较次数为

$$\begin{aligned} C &= 0.20 \cdot 1 + (0.22 + 0.25) \cdot 2 \\ &\quad + (0.18 + 0.05 + 0.08) \cdot 3 + 0.02 \cdot 4 \\ &= 0.20 + 0.94 + 0.93 + 0.08 = 2.15. \end{aligned}$$

补充推导：上述树由 S9 区间最优根恢复，并直接验证成本。若每个区间都贪心选概率最大者为根，根为 return，所得成本为 2.43；若使用以 do 为根、左右分别以 case 和 switch 为根的完全平衡七节点树，成本为 2.70。这说明频率局部贪心和高度平衡不能代替概率加权优化。特定 AVL 旋转结果依赖插入顺序，不能仅从概率表唯一确定。

普通 DP 时间为 $O(N^3)$ ，表空间为 $O(N^2)$ 。S9 还指向教材的 $O(N^2)$ 优化算法，作为延伸阅读，不在此展开证明。[S9]

4. 全源最短路径 (All-Pairs Shortest Paths) [S10–S11]

全源最短路径 (All-Pairs Shortest Paths, APSP) 求所有顶点对之间的最短距离。这里以允许哪些中间顶点为阶段，而不是以路径边数为阶段。[S10]

4.1 重复单源与阶段状态 (Repeated Single Source and Stage State) [S10]

重要性：A · 必须掌握

方法一对每个顶点运行一次单源最短路径 (Single-Source Shortest Path) 算法；方法二直接构造所有顶点对的阶段表。[S10]

解释补充：重复单源复杂度取决于算法。邻接矩阵的朴素 Dijkstra 每次 $O(N^2)$ ，重复 N 次为 $O(N^3)$ ；邻接表配合堆在非负权稀疏图上可更快。不能认为所有单源算法重复运行都恰好 $O(N^3)$ ，也不能在负边图上无条件使用 Dijkstra。

顶点编号 $0, \dots, N-1$ ，定义

$D_k[i][j]$ = 仅允许 $\{0, 1, \dots, k\}$ 为中间顶点时， $i \rightarrow j$ 的最短距离。

初始 $D_{-1}[i][j] = A[i][j]$ 。邻接矩阵 (Adjacency Matrix) A 对角为 0，无直接边时为 $+\infty$ ；最终答案为 D_{N-1} 。[S10–S11]

4.2 不经过 k 与经过 k (Without and Through Vertex k) [S10]

重要性：A · 必须掌握

从允许 $0, \dots, k-1$ 扩展到允许 $0, \dots, k$ ，分类为：

1. 不使用 k 为中间点：候选仍是 $D_{k-1}[i][j]$ 。
2. 使用 k 为中间点：拆成 $i \rightarrow k$ 与 $k \rightarrow j$ ，候选是 $D_{k-1}[i][k] + D_{k-1}[k][j]$ 。
3. 比较两类：

$$D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j]).$$

允许 k 并不等于强制经过 k 。[S10]

核心原理 (Core Principle)

无负环时可选取没有重复顶点的最短路径；若它经过新允许的 k ，就能在 k 处分成两个只使用旧中间顶点的最优子路径。

1. 某段子路径若不是最短，则替换后会让总路径更短，违反最优性。
2. 两段其他中间顶点来自旧集合，故可读取上一阶段结果。
3. 两类候选覆盖全部可能，取最小得到当前阶段。[S10；证明展开为补充]

4.3 Floyd-Warshall 实现与恢复 (Implementation and Reconstruction) [S11]

重要性：A · 必须掌握

Floyd-Warshall 算法 (Floyd-Warshall Algorithm) 的循环顺序先 k 、再 i 、再 j ：

```
D = A
for k = 0 .. N-1:
    for i = 0 .. N-1:
        for j = 0 .. N-1:
            if D[i][k] and D[k][j] are finite:
                candidate = D[i][k] + D[k][j]
                if candidate < D[i][j]:
                    D[i][j] = candidate
```

1. k 在最外层：每次完成一个新增允许中间点的阶段，保持状态含义。
2. 有负边但无负环：仍可求有限的最短距离。
3. 存在负环：若结束后某个 $D[i][i] < 0$ ，说明有负环；可经过该负环的路径对没有有限最短值。[S11]

核心原理 (Core Principle)

无负环时对角距离为零，当前阶段对 $D[i][k]$ 与 $D[k][j]$ 的更新不会使它们更小，因此可用同一矩阵原地转移。

补充推导：对 $D[i][k]$ 的候选为 $D[i][k] + D[k][k] = D[i][k]$ ，对 $D[k][j]$ 同理，所以读取这两项仍等价于读取上一阶段。此解释以无负环为前提；有负环时只能使用检测结论，不能把最终矩阵认作所有点对的有限最优值。

时间为 $O(N^3)$ ，距离矩阵空间为 $O(N^2)$ 。通常适合稠密图；与重复单源的实际比较还取决于图表示与单源算法。[S11]

解释补充： 有限整数哨兵表示无穷大时，应先检查可达性再相加，防止溢出或“无穷大 + 负数”误判。若输入存在负自环，也应保留负权信息，不能初始化时无条件覆盖成 0。

路径恢复可设 $\text{Next}[i][j]$ ：有直接边时记为 j ；通过 k 严格改进时改为 $\text{Next}[i][k]$ ；从 i 反复按 $\text{Next}[\text{current}][j]$ 前进即可恢复路径。结构占 $O(N^2)$ 空间，只对存在有限最短路径的点使用。[S11 的恢复提示；具体方法为解释补充]

5. 动态规划的四个步骤（Four Steps of DP Design）[S12]

重要性：A · 必须掌握

1. 刻画最优解结构（Characterize an Optimal Solution）：说明最后一个决策怎样划分子问题，子问题为什么必须最优。
2. 递归定义最优值（Recursively Define Optimal Values）：明确状态、边界和转移。
3. 按依赖顺序计算（Compute Values in Order）：确定每个状态所需的已完成状态和填表顺序。
4. 恢复方案（Reconstruct the Strategy）：求值时记录决策，再从终点反向还原。[S12]

解释补充： 求最优值与输出最优方案不同。矩阵链记录分割点，OBST 记录根，最短路径记录后继，装配线记录前驱线路；只存成本可能无法直接恢复这些选择。

6. 双装配线（Two-Line Product Assembly）[S13–S16]

两条生产线都能完成同一辆车，每阶段加工时间可能不同，阶段间允许换线，目标是最小化总装配时间。[S13]

6.1 搜索空间与最优子结构（Search Space and Optimal Substructure）[S13–S14]

重要性：A · 必须掌握

N 个阶段各选两条线之一，完整线路序列有 2^N 个。S13 用 $O(2^N)$ 时间与 $O(N)$ 当前路径空间描述穷举的指数级搜索。[S13]

解释补充： DFS 增量累计成本可用指数级总工作枚举；若每个完整方案从头扫描 N 阶段计算成本，还会多一个 N 因子，两者均为指数增长。

核心原理（Core Principle）

若最优路径到达某阶段、某线路之前的前缀不是该状态的最优前缀，就能换成更快的前缀并保持后续不变，得到更小总时间。

1. 状态必须同时记录阶段和所在的线路。

2. 到达相同状态的不同路径具有相同的后续选择，只需保留较快者。
3. 替换前缀不影响未来加工和换线选择，所以最优路径包含该状态的最优前缀。[S14；证明展开为补充]

6.2 留线与换线转移 (Stay and Switch Transitions) [S15]

重要性：A · 必须掌握

令 $f[\ell][s]$ 表示到达阶段 s 、线路 $\ell \in \{0, 1\}$ 的最小累计时间，初始 $f[0][0] = f[1][0] = 0$ 。原代码的表达是：

$$f[\ell][s] = \min(f[\ell][s-1] + t_{\text{process}}[\ell][s-1], f[1-\ell][s-1] + t_{\text{transit}}[1-\ell][s-1]).$$

按原顺序分类：

1. **留在当前线：**来自 ℓ 的上一阶段，使用 t_{process} 。
2. **从另一线切入：**来自 $1-\ell$ 的上一阶段，使用 t_{transit} 。
3. **终点：**取 $\min(f[0][N], f[1][N])$ 。[S15]

解释补充：只有当 t_{transit} 是从另一线上一阶段到本阶段的完整转移成本、已含加工时，上式才不会漏计加工。若它只是额外换线时间，第二项还必须加上目标工位加工时间。S13-S15 没有给出可确认的完整定义，不能把换线项不加加工当作普遍规则。

核心原理 (Core Principle)

当前线路只有两种可能前驱；分别用最优前缀加完整转移成本，比较后得到当前最优值。

采用明确的“工位加工 + 额外换线”约定： $a_{\ell,s}$ 为阶段 s 的加工时间， $\tau_{\ell,s-1}$ 为从线路 ℓ 的上一阶段切换到另一线的额外时间。此时

$$f_{\ell,s} = a_{\ell,s} + \min(f_{\ell,s-1}, f_{1-\ell,s-1} + \tau_{1-\ell,s-1}).$$

补充推导：上式把额外换线与目标加工分开，说明原程序第二项的语义要求，并非原变量定义的逐字转录。如果还有入线、出线成本，应分别纳入初始状态和终点比较；本课件没有给这些输入。

按阶段递增，每阶段两个状态，每状态两个候选，所以时间 $O(N)$ 、完整表空间 $O(N)$ 。[S15]

解释补充：只求值可滚动保存上一阶段两项，辅助空间为 $O(1)$ 。必须从同一份旧状态计算两个新值再一起更新；不能先覆盖一条线的旧值，让另一条线错误读取当前阶段。

6.3 前驱记录与方案回溯 (Predecessors and Reconstruction) [S16]

重要性：A · 必须掌握

$L[\ell][s]$ 记录最优状态来自的上一阶段线路：

1. **f_stay < f_move**: 保留留线成本, 记 $L[\ell][s] = \ell$ 。
2. **否则**: 保存换线成本, 记 $L[\ell][s] = 1 - \ell$ 。平局时原代码选择此分支, 仍是一条最优路线。[S16]

核心原理 (Core Principle)

每个最优状态记录前驱, 从最佳终点沿前驱倒着走, 就恢复一条完整最优路线。

1. 终点选择更小成本的线路, 原代码平局时选线路 1。
2. 从 $s = N$ 倒数到 1, 先保存 $\text{plan}[s] = \text{line}$ 。
3. 再更新 $\text{line} = L[\text{line}][s]$ 回到上一阶段。
4. 按阶段递增输出 plan 。[S16]

回溯时间 $O(N)$ 。前驱表与完整方案仍占 $O(N)$ 空间; 即使最优值计算采用滚动, 也不能把含方案恢复的总空间写成 $O(1)$ 。[S16; 空间区别为解释补充]

7. 动态规划要素与项目 (DP Elements and Project) [S17–S19]

7.1 最优子结构与重叠 (Optimal Substructure and Overlap) [S17]

重要性: A · 必须掌握

两个关键要素是最优子结构 (Optimal Substructure) 与重叠子问题 (Overlapping Subproblems)。[S17]

对不能直接使用当前 DP 的情况分类:

1. **子解不能独立替换**: 局部更优会改变后续可行性, 需要重新建模。
2. **状态遗漏相关历史**: 两条看似同状态的路径实际上有不同未来选择, 不能直接丢掉其中一条。
3. **没有重叠子问题**: 分治仍可求解, 但存表未必节省工作。
4. **状态或转移过多**: DP 即使成立, 也可能不具备可用复杂度。[S17; 分类为解释补充]

解释补充: 有时增加状态信息能恢复最优子结构, 但扩大后的空间可能是指数级。Fibonacci 不是最优问题, 也能用保存重复状态答案的同一思路。

7.2 美丽子序列 (Beautiful Subsequence) [S18]

重要性: C · 了解即可

给定整数 m , 至少有两个元素的序列若存在一对相邻元素, 其差不超过 m , 则按题意称为美丽; 要求计算输入整数序列中的美丽子序列数量。完整要求在 Pintia 项目中。[S18]

建模先区分:

1. **子序列 (Subsequence)**: 保持原相对顺序, 下标可以不连续。
2. **子序列中的相邻元素**: 在选出后的序列中相邻, 未必在原数组中相邻。
3. **存在一对满足条件**: 不等于每一对相邻元素都满足条件。[S18; 语义展开为解释补充]

解释补充：可考虑“所有子序列数减去不满足条件的子序列数”，但这只是建模方向。状态与数据结构取决于值域、规模、取模和差的定义，当前课件不足以确认最终算法。

7.3 参考阅读 (Further Reading) [S19]

《Introduction to Algorithms》第 3 版第 15 章，课件页码 359–413。[S19]

本讲总结 (Lecture Summary)

- DP 保存重复子问题的答案；只依赖前两个状态时，可滚动压缩空间。[S2–S3]
- 矩阵链按最后一次乘法切分连续区间，按长度填表，记录分割点恢复括号。[S4–S6]
- OBST 按根切分区间，子树深度加一，新增成本为区间概率和。[S7–S9]
- Floyd–Warshall 限制中间顶点集合， k 必须放在最外层。[S10–S11]
- DP 设计依次明确结构、转移、顺序和恢复，不能只写递推式。[S12]
- 装配线每状态只有两个前驱，每个转移仍要包含完整成本。[S13–S16]
- 状态要足以描述未来，且数量可控。[S17]

复杂度与条件速查 (Complexity and Conditions)

算法	状态与转移	时间	辅助空间	条件与来源
朴素 Fibonacci	重复递归	$\Theta(\varphi^N)$	$O(N)$ 栈	固定成本算术；S2–S3、补充
滚动 Fibonacci	每下标一次	$O(N)$	$O(1)$	只求最终值；S3
矩阵链 DP	$O(N^2)$ 区间，各枚举 $O(N)$ 分割	$O(N^3)$	$O(N^2)$	尺寸兼容、顺序固定；S5–S6
普通 OBST DP	$O(N^2)$ 区间，各枚举 $O(N)$ 根	$O(N^3)$	$O(N^2)$	成功查询概率固定；S7–S9
Floyd–Warshall	N 阶段，每阶段 N^2 对	$O(N^3)$	$O(N^2)$	有限最短值要求无负环；S10–S11
装配线 DP	$2N$ 状态，每个两个前驱	$O(N)$	$O(N)$	含完整表或方案恢复；S15–S16
装配线滚动值	同上	$O(N)$	$O(1)$	仅最优值，不含方案；补充

易错点 (Common Pitfalls)

1. 把本讲 Fibonacci 的 $F(0)$ 写成 0，导致例值错位。
2. 认为 DP 一定不能使用递归，忽略记忆化的等价思路。
3. 矩阵链允许重新加括号，不允许交换矩阵。
4. 合并成本误用 r_i ；左结果行数是 r_{i-1} 。
5. OBST 混淆空树、单节点与不同深度成本约定。
6. 用 AVL 高度平衡代替概率加权最优。
7. Floyd 的 k 放进内层，破坏阶段不变量。
8. 无穷大哨兵参与加法，或在负环存在时仍声称有限最短答案。
9. 装配线只加额外换线时间，漏掉目标工位加工。

10. 只存最优值，没有记录决策，却要求直接恢复方案。

复习自测 (Self-Check)

1. 为什么 $F(6)$ 重复计算 $F(4)$? 滚动变量在每轮前表示什么? [S2-S3]
2. 两个四矩阵顺序为什么分别需要 125000 与 2200 次乘法? [S4]
3. 最后一次矩阵乘法为何对应连续区间分割? [S5]
4. 任意顺序填区间表会在哪类依赖上失败? [S6]
5. OBST 新增根时为什么新增区间概率和? [S8]
6. 用树中各键深度验证最优成本 2.15。[S9]
7. Floyd 的两个候选分别读取什么上一阶段状态? [S10]
8. 为什么能原地更新? 为何需要无负环条件? [S11]
9. 最优值与最优方案分别需要保存什么? [S12、S16]
10. t_{transit} 若只是额外换线时间，转移式怎样修正? [S15]
11. 滚动计算两条线时为何不能先覆盖一条旧状态? [S15]
12. 两条路径到达同阶段但不同线路时，能否只保留更快的一条? [S14、S17]

术语速查 (Terminology)

中文	English	含义
动态规划	Dynamic Programming, DP	保存并复用子问题答案
重叠子问题	Overlapping Subproblems	相同状态在多个分支出现
最优子结构	Optimal Substructure	最优解包含对应状态的最优子解
循环不变量	Loop Invariant	每次迭代保持的关系
记忆化	Memoization	递归中缓存已求状态
矩阵链乘法	Matrix-Chain Multiplication	优化有序链括号化
括号化	Parenthesization	指定结合顺序
Catalan 数	Catalan Number	计数完整二元括号化等结构
最优二叉搜索树	Optimal Binary Search Tree, OBST	最小化固定概率下期望查询成本
全源最短路径	All-Pairs Shortest Paths, APSP	求全部顶点对最短路径
负环	Negative Cycle	总边权为负的环
前驱	Predecessor	最优状态对应的上一步
子序列	Subsequence	原顺序选取, 不要求连续

待核对 (Items to Verify)

- [待核对 S5、S8] 部分递推、权重求和和最优根条件在原生提取及定点渲染中未显示；笔记依据状态和后续代码补全，不能认作逐字转录。
- [待核对 S8] 原文 $c_{ii} = 0$ 与同页根成本、S9 单节点表值不一致。笔记统一为空区间 0、单节点 p_i 的比较次数模型，应对照教师完整版确认。
- [待核对 S13–S15] t_{transit} 是否包含加工成本没有可靠定义。笔记保留原式，给出两种语义和不漏计加工的补充式；实现前需核对项目输入。
- [待核对 S18] 完整输入规模、差是否为绝对差、是否按下标区分子序列及取模要求未提供，应以 Pintia 项目说明为准，不能擅自确定最终算法。